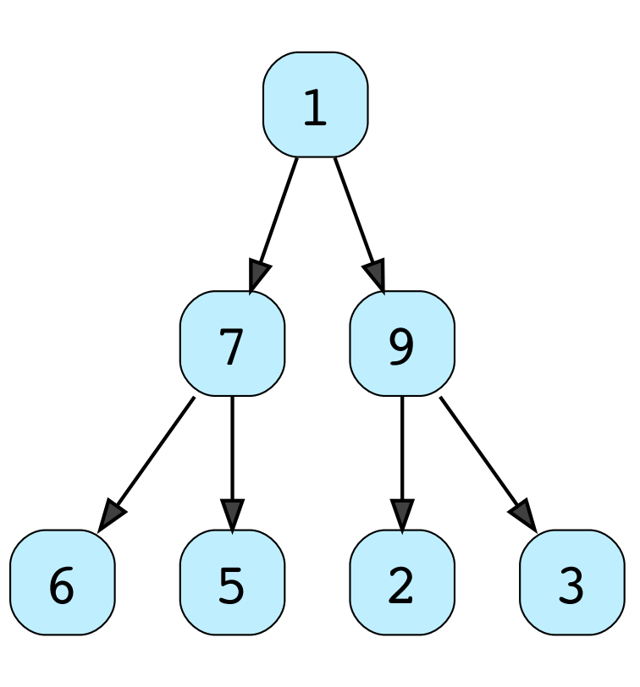
Solution: Count Paths for a Sum

Problem Statement

Given a binary tree and a number ‘S’, find all paths in the tree such that the sum of all the node values of each path equals ‘S’. Please note that the paths can start or end at any node but all paths must follow direction from parent to child (top to bottom).

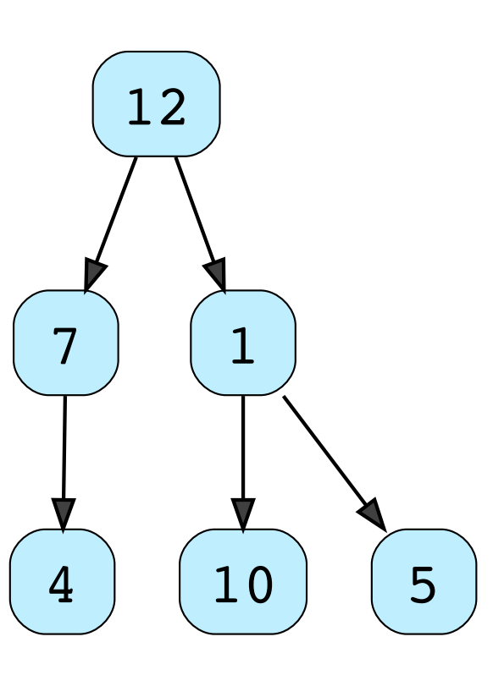
Example 1:

S: 12
Output: 3
Explanation: There are three paths with sum '12':
7 -> 5, 1 -> 9 -> 2, and 9 -> 3



Example 2:

S: 11
Output: 2
Explanation: Here are the two paths with sum '11':
7 -> 4 . and 1 -> 10.

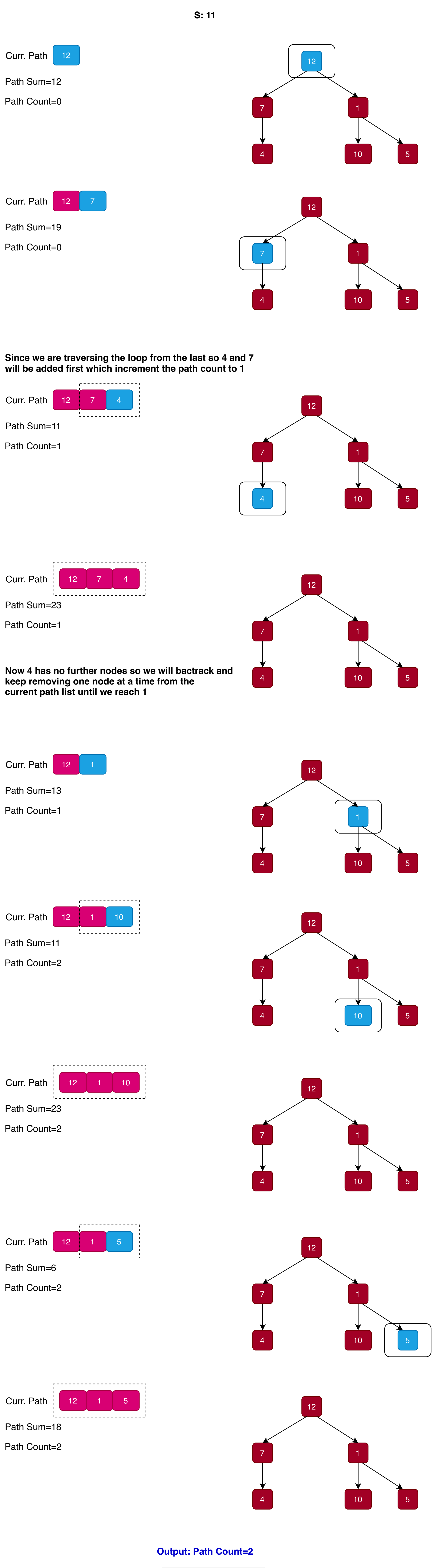


Solution

This problem follows the **Binary Tree Path Sum** pattern. We can follow the same DFS approach. But there will be four differences:

1. We will keep track of the current path in a list which will be passed to every recursive call.
2. Whenever we traverse a node we will do two things:
3. Add the current node to the current path.
4. As we added a new node to the current path, we should find the sums of all sub-paths ending at the current node. If the sum of any sub-path is equal to ‘S’ we will increment our path count.
5. We will traverse all paths and will not stop processing after finding the first path.
6. Remove the current node from the current path before returning from the function. This is needed to Backtrack while we are going up the recursive call stack to process other paths.

Here is the visual representation of the algorithm:



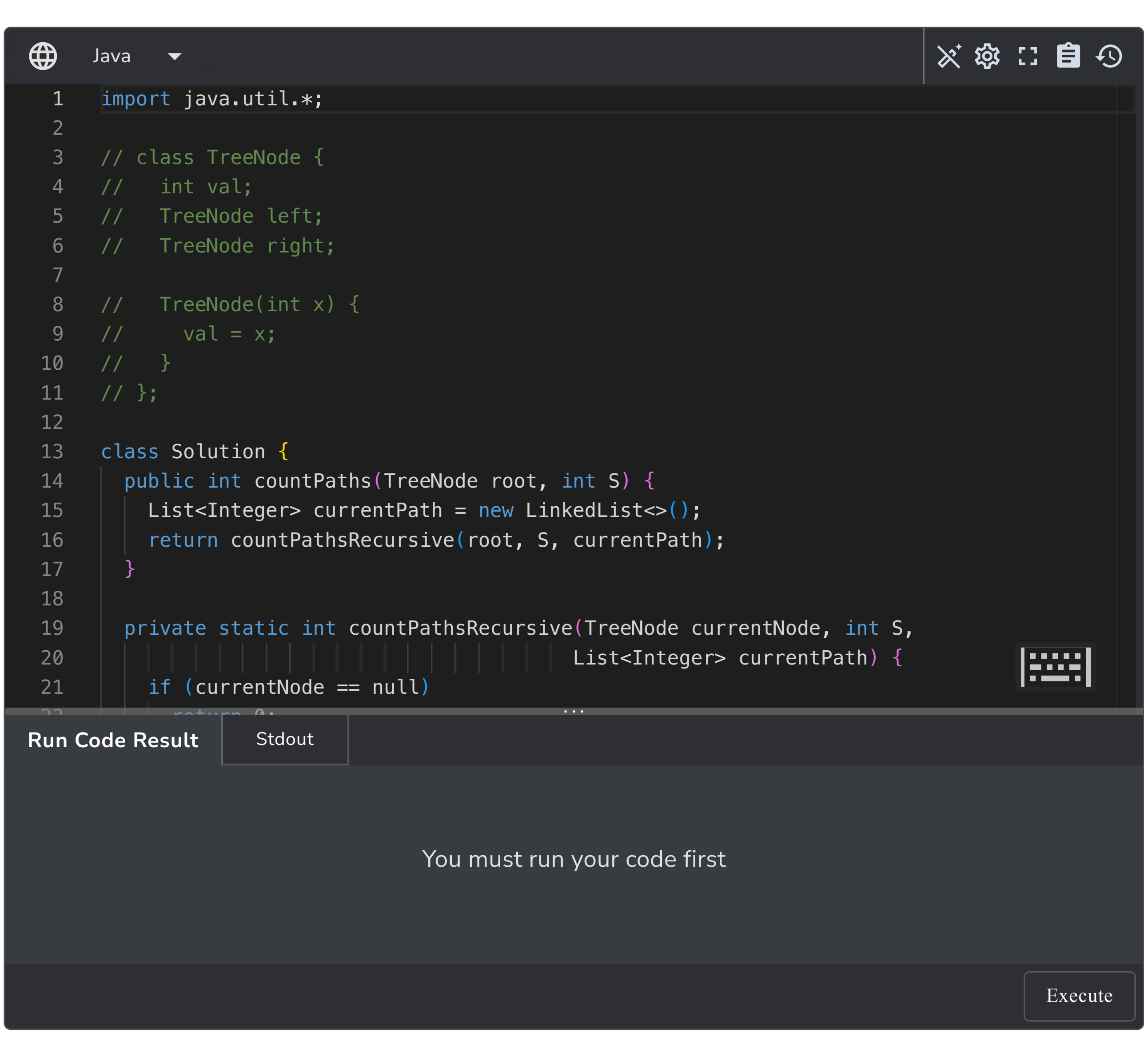
Since we are traversing the loop from the last so 4 and 7 will be added first which increment the path count to 1

Now 4 has no further nodes so we will bactrack and keep removing one node at a time from the current path list until we reach 1

Count Paths for a Sum

Code

Here is the code for this algorithm:



```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public int countPaths(TreeNode root, int S) {
    List<Integer> currentPath = new LinkedList<>();
    return countPathsRecursive(root, S, currentPath);
  }

  private static int countPathsRecursive(TreeNode currentNode, int S,
                                         List<Integer> currentPath) {
    if (currentNode == null)
```

Time Complexity

The time complexity of the above algorithm is $O(N^2)$ in the worst case, where ‘N’ is the total number of nodes in the tree. This is due to the fact that we traverse each node once, but for every node, we iterate the current path. The current path, in the worst case, can be O(N) (in the case of a skewed tree). But, if the tree is balanced, then the current path will be equal to the height of the tree, i.e., O(logN). So the best case of our algorithm will be O(NlogN).

Space Complexity

The space complexity of the above algorithm will be O(N). This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child). We also need O(N) space for storing the `currentPath` in the worst case.

Overall space complexity of our algorithm is O(N).

A more efficient solution

Can we further improve the solution?

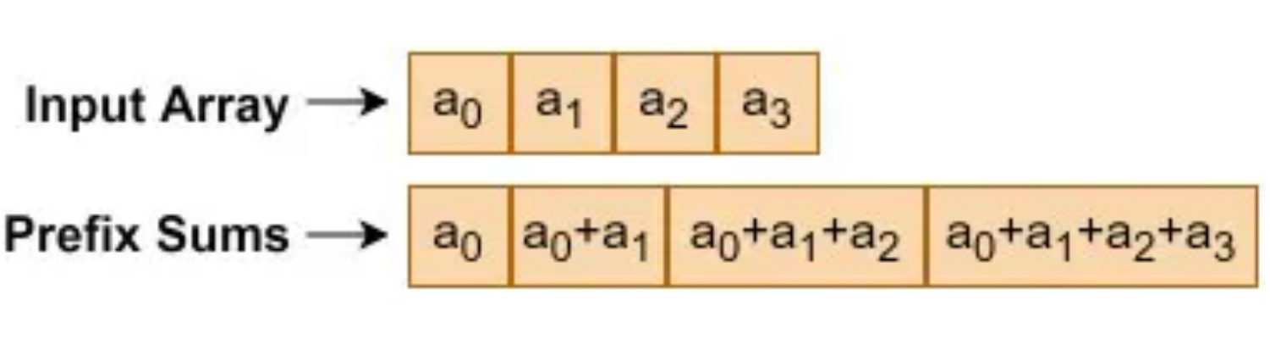
One thing we are repeating for each node is traversing the current path and seeing if any sub-path that ends at the current node gives us the required sum.

Let’s see if we can improve this part.

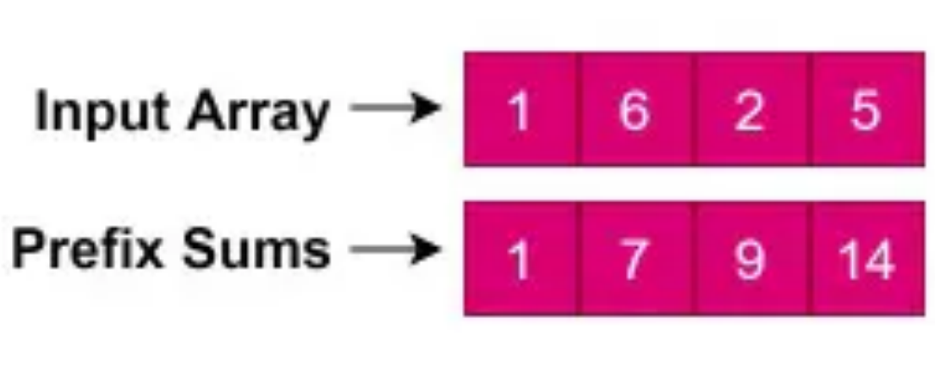
We can use the Prefix Sum technique to efficiently manage the path sums.

Prefix Sum

Let’s first understand what **Prefix Sum** is. For a given array, its Prefix Sum is another array where each element is the commutative sum of the corresponding element in the given array and all its previous elements.



Here is an example:



Now, let’s say we want to find all subarrays of a given array with a target sum.

Let’s say our target sum is `7`, and we want to find all the subarrays of the array mentioned above.

We can clearly see that there are two such subarrays: 1) `[1, 6]`, and 2) `[2, 5]`.

How can we utilize the Prefix Sum array to find these two subarrays efficiently?

There are two ways Prefix Sum can help us:

a) Since each element of the prefix sum array contains the cumulative sum of current and previous elements, therefore, whenever we see our target sum, we have found our targeted subarray. For example, since the second element of the prefix sum array is 7; therefore, our target subarray will be from the start of the array till the second element, i.e., `[1, 6]`

(**b**) Secondly, the prefix sum array can also help us find our target subarray that is not starting from the first index.

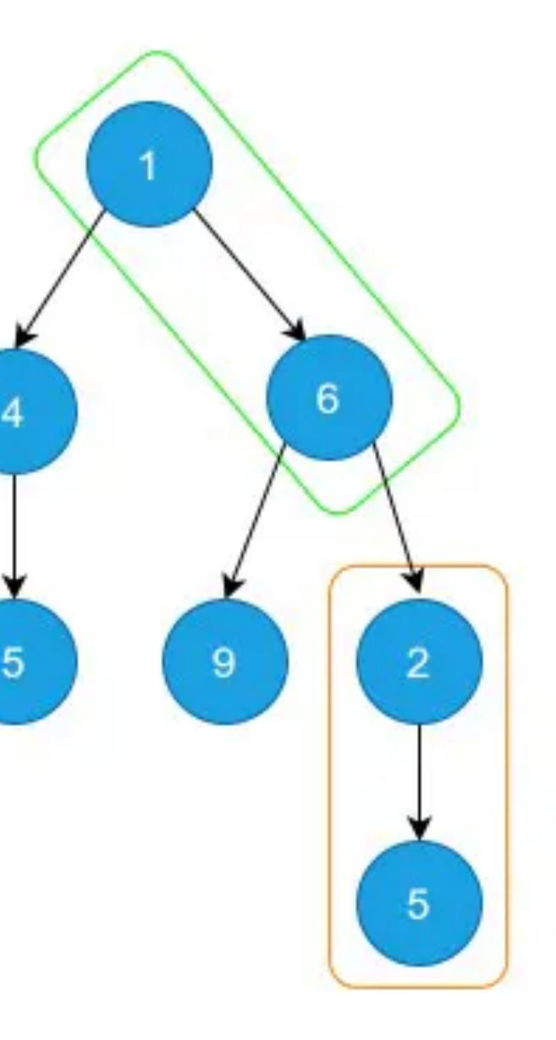
If we subtract the target sum from any element of the prefix sum array, the result will also give us our target subarray (if that result is present in the prefix sum array).

For example, take the 4th element of the prefix sum array and subtract the target sum from it: `14 - 7 => 7`

Is this result (`7`) present in the prefix sum array? Yes, it is the second element. This means the sum from the 3rd element to the current element (i.e., the 4th) is also `7`.

Hence, our target subarray will be from the 3rd element to the current element, i.e., `[2, 5]`.

Now, let’s see how we can use prefix sum for binary trees. Take the following example:

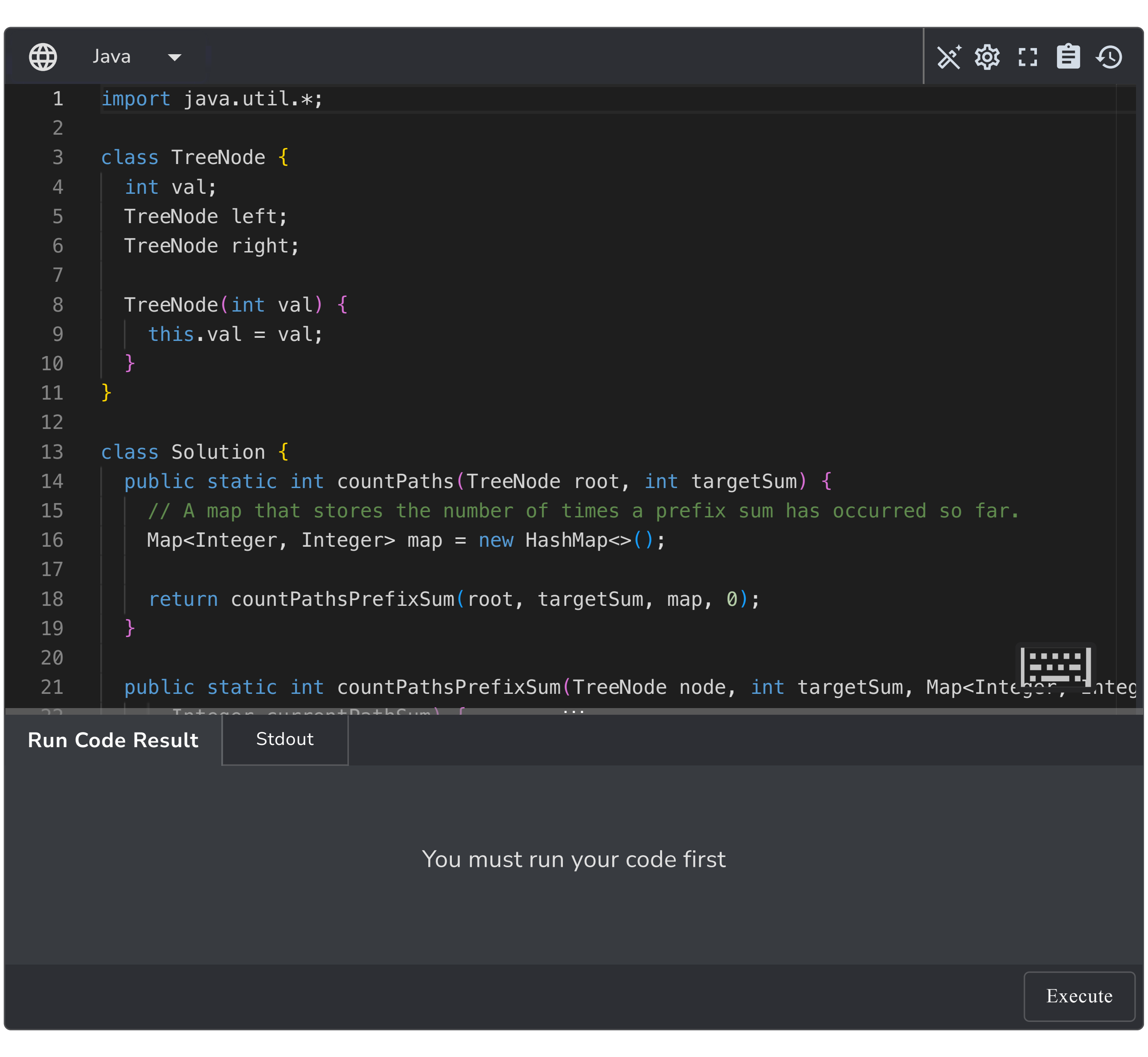


We can consider each path as an array and calculate its prefix sums to find any required sub-paths. In the above tree, the highlighted sub-paths are exactly the same as our previous array example.

Here is what our new algorithm will look like:

Code

Here is the code for this algorithm:



```java
import java.util.*;

class TreeNode {
  int val;
  TreeNode left;
  TreeNode right;

  TreeNode(int val) {
    this.val = val;
  }
}

class Solution {
  public static int countPaths(TreeNode root, int targetSum) {
    // A map that stores the number of times a prefix sum has occurred so far.
    Map<Integer, Integer> map = new HashMap<>();

    return countPathsPrefixSum(root, targetSum, map, 0);
  }

  public static int countPathsPrefixSum(TreeNode node, int targetSum, Map<Integer, Integ
```

Time Complexity

As we are not traversing the current path for each node, the time complexity of the above algorithm will be O(N) in the worst case, where ‘N’ is the total number of nodes in the tree.

Space Complexity

The space complexity of the above algorithm will be O(N). This space will be used to store the recursion stack, as well as for the prefix sum.

← Previous
Count Paths For A Sum (Medium)

Next →
Problem Challenge 1: Tree Diameter (Medium)

☐ Mark as Completed